NAME: Jeevitha.S
PROJECT TOPIC: Food Delivery Management System

Assignment 3: Database Design Analysis

DBMS Coursework | Normalization, Functional Dependencies & Anomaly Mitigation

1. Theoretical Context: Poor Design & Anomaly Mechanics

In relational database design, poor architecture typically arises from bad schema decomposition, unnormalized structures, or missing integrity constraints. When attributes with independent real-world lifecycles are combined into a single flat table, excessive data redundancy occurs. This redundancy leads directly to three critical operational anomalies during data modification:

Insertion Anomaly

Occurs when data regarding an entity cannot be recorded without artificially tying it to an unrelated entity. For instance, if restaurant details and menu items were stored in a single table, a newly registered restaurant could not be added unless it immediately created at least one menu item (if Primary Key requires an Item ID).

Deletion Anomaly

Occurs when deleting a row to remove details of one entity unintentionally destroys essential data about a completely different entity. For example, deleting the only customer order containing a specific promotional food item could permanently erase that food item's entry from the system database.

2. Functional Dependencies (FDs) Across System Entities

A Functional Dependency (`X → Y`) states that the value of attribute X uniquely determines the value of Y. The core functional dependencies identified across the 8 schema entities of our system are summarized below:

Entity Table	Determinant (X)	Dependent Attributes (Y)
Customer	`Customer_ID`	Name, Phone, Email, Address
Restaurant	`Restaurant_ID`	Restaurant_name, Phone, Address, Rating
Menu_Item	`Item_ID`	Restaurant_ID, Item_name, Category, Price, Availability

Order	`Order_ID`	Customer_ID, Restaurant_ID, Order_Date, Total_amount, Order_status
Order_Item	`Order_item_ID`	Order_ID, Item_ID, Quantity, Unit_price
Order_Item (Alt Key)	`{Order_ID, Item_ID}`	Quantity, Unit_price
Payment	`Payment_ID`	Order_ID, Payment_method, Payment_status, Payment_date, Amount
Delivery_Partner	`Partner_ID`	Name, Phone, Vehicle_type, Partner_status
Delivery	`Delivery_ID`	Order_ID, Partner_ID, Pickup_Time, Delivery_Time, Delivery_Status

3. Design Analysis & Identification of Redundancies

Evaluating the proposed 8-table schema (Customer, Restaurant, Menu_Item, Order, Order_Item, Payment, Delivery_Partner, Delivery) demonstrates that the design successfully eliminates major normalization issues found in unnormalized system models.

Analysis of Denormalized Alternative vs. Proposed Normalized Design

Scenario: If we had combined *Order* and *Order_Item* into a single unnormalized table:

• **Redundancy:** Customer_ID, Restaurant_ID, and Order_Date would repeat for every food item ordered in a single transaction.
• **Update Anomaly:** Changing an Order_Status from "Preparing" to "Out for Delivery" would require updating N rows for an order with N items.
• **Resolution in Proposed Model:** Splitting into `Order` (header data) and `Order_Item` (line items) enforces 3NF and eliminates structural repetition.

4. Normalization Breakdown (1NF to BCNF)

The proposed database design adheres strictly to relational normal forms as evaluated below:

Normal Form	Requirement Criteria	Compliance Status	Result
1NF	All columns contain atomic (indivisible) values; no repeating groups or arrays.	Satisfied: Each column holds single scalar values (e.g., single phone number, single price).	**PASS**
2NF	Must be in 1NF and all non-key attributes are fully functionally dependent on the primary key (no partial dependencies).	Satisfied: All tables use single-attribute surrogate PKs (or clear composite determinants) where all attributes depend on the entire key.	**PASS**
3NF	Must be in 2NF and no transitive dependencies exist (non-key attribute determines another non-key attribute).	Satisfied: All attributes depend directly on the primary key (e.g., Customer Address depends on Customer_ID, not Phone).	**PASS**
BCNF	For every functional dependency X → Y, X must be a super key.	Satisfied: All functional determinants in every relation are candidate/primary keys.	**PASS**

5. Proposed Schema Improvements & Architecture Refinements

To transition the existing logical design into an enterprise-ready production schema, the following structural enhancements are recommended:

1. Separate Address Management Entity

Extract repeated address strings into a dedicated `Customer_Address` table (`Address_ID, Customer_ID, Street, City, Zip, Is_Primary`) to support multiple delivery locations per customer and reduce string duplication.

2. Historical Price Preservation

Maintain `Unit_price` in `Order_Item` independently of `Menu_Item.Price`. This ensures menu price edits in the future do not retroactively alter historical revenue statistics in past orders.

3. Explicit Constraints & Unique Indices

Apply `UNIQUE` constraints on `Customer.Email`, `Customer.Phone`, `Delivery.Order_ID` (to enforce 1:1 relationship), and `Payment.Order_ID` (to prevent duplicate payments for a single order).